

Bài giảng: Quản lý vòng đời Table trong MySQL

CREATE TABLE, AUTO_INCREMENT, ALTER, RENAME, DROP, TEMPORARY,
TRUNCATE & Generated Columns

Vũ Đức Minh

Trường Đại học Kinh tế Quốc dân (NEU), Hà Nội, Việt Nam

September 23, 2026

Mục tiêu bài giảng

Sau khi hoàn thành bài học này, người học có thể:

- Nắm vững chuỗi **vòng đời bảng (Table Lifecycle)** từ khởi tạo đến loại bỏ trong CSDL.
- Vận dụng các tùy chọn **CREATE TABLE** nâng cao: IF NOT EXISTS, LIKE, AS SELECT.
- Sử dụng **AUTO_INCREMENT** chuẩn xác để tạo numeric surrogate key và dùng LAST_INSERT_ID().
- Khảo sát chi tiết siêu dữ liệu với **SHOW CREATE TABLE, SHOW COLUMNS, DESCRIBE, SHOW INDEX**.
- Thay đổi cấu trúc bảng bằng **ALTER TABLE**: thêm, xóa, đổi tên, sửa kiểu cột (MODIFY vs CHANGE).
- Đổi tên bảng an toàn với **RENAME TABLE** (kỹ thuật atomic swap và chuyển đổi schema).
- Phân biệt sự khác nhau bản chất giữa **TRUNCATE TABLE** và **DELETE**

Nội dung bài học

Phần 1: Khởi tạo & Khảo sát

- 1. Vòng đời Table & Rủi ro DDL ngầm định
- 2. CREATE TABLE: Options, LIKE & CTAS
- 3. AUTO_INCREMENT & LAST_INSERT_ID()
- 4. Kiểm tra cấu trúc Metadata

Phần 2: Tiến hóa, Xóa & Cột tự sinh

- 5. ALTER TABLE: ADD, DROP, MODIFY, CHANGE
- 6. RENAME TABLE & DROP TABLE an toàn
- 7. Bảng tạm: CREATE TEMPORARY TABLE
- 8. TRUNCATE TABLE vs DELETE FROM
- 9. Generated Columns: VIRTUAL vs STORED
- 10. Checklist Migration Production & Tổng kết

1. Chuỗi vòng đời của Table trong RDBMS

Thiết kế (Schema Design) → Xác định thực thể, kiểu dữ liệu, quan hệ & ràng buộc

Khởi tạo (CREATE TABLE) → Tạo cấu trúc bảng vật lý trong InnoDB

Vận hành DML → Thêm, sửa, xóa, truy vấn dữ liệu (INSERT, UPDATE, DELETE, SELECT)

Tiến hóa (ALTER TABLE / RENAME) → Thêm cột, sửa kiểu, đánh chỉ mục theo nghiệp vụ mới

Dọn dẹp / Tạm thời (TEMPORARY / TRUNCATE) → Xử lý trung gian hoặc làm trống nhanh

Loại bỏ (DROP TABLE) → Hủy cấu trúc khi hết vòng đời lưu trữ (retention period)

2. Bản chất DDL: Khác biệt sống còn với DML

DML (Data Manipulation)

- Các lệnh: INSERT, UPDATE, DELETE.
- Tác động trực tiếp lên dữ liệu dòng.
- **Hỗ trợ Transaction đầy đủ:** Có thể dùng ROLLBACK để khôi phục trạng thái ban đầu nếu xảy ra lỗi.

DDL (Data Definition)

- Các lệnh: CREATE, ALTER, DROP, RENAME, TRUNCATE.
- Thay đổi cấu trúc và từ điển dữ liệu (data dictionary).
- **Gây Implicit Commit ngầm định:** Mọi transaction mở trước đó sẽ bị ép commit ngay lập tức!
- **Không thể Rollback** trong phiên giao dịch thông thường!

Nguyên tắc vận hành: Trước khi chạy bất kỳ câu lệnh DDL nào trên Production, bắt buộc phải có backup định nghĩa và dữ liệu!

3. Các dạng khởi tạo bảng: LIKE vs AS SELECT

CREATE TABLE ... LIKE

- Sao chép **toàn bộ cấu trúc khung (skeleton)**, bao gồm kiểu cột, khóa chính, index, engine.
- Bảng tạo ra hoàn toàn **rỗng (0 dòng)**.

```
CREATE TABLE products_backup  
LIKE products;
```

CREATE TABLE ... AS SELECT

- Tạo bảng mới và **đổ luôn dữ liệu** từ kết quả câu truy vấn.
- **Cảnh báo:** Không sao chép các thuộc tính khóa chính, AUTO_INCREMENT, foreign keys hoặc secondary index!

```
CREATE TABLE products_copy AS  
SELECT product_id, product_name, unit_price  
FROM products  
WHERE unit_price > 500000;
```

CREATE TABLE IF NOT EXISTS

4. AUTO_INCREMENT: Cơ chế Surrogate Key tự sinh

Quy tắc thiết kế AUTO_INCREMENT

- Mỗi bảng **chỉ có duy nhất một cột** mang thuộc tính AUTO_INCREMENT.
- Cột đó bắt buộc phải thuộc kiểu số nguyên và **phải được đánh chỉ mục (thường là Primary Key)**.
- Cột AUTO_INCREMENT không thể có giá trị DEFAULT.

```
CREATE TABLE suppliers (
  supplier_id INT UNSIGNED AUTO_INCREMENT
  ,
  supplier_name VARCHAR(120) NOT NULL,
  PRIMARY KEY (supplier_id)
) ENGINE = InnoDB;

-- Chen khong can dua ID:
INSERT INTO suppliers (supplier_name)
VALUES ('Alpha Tech'), ('Beta Logistics');

-- Lay ID vua sinh trong session hien tai:
SELECT LAST_INSERT_ID();
```

Phân biệt LAST_INSERT_ID() vs MAX(id)

LAST_INSERT_ID() an toàn tuyệt đối trong môi trường đa kết nối (concurrent

5. Khảo sát Metadata cấu trúc bảng

4 câu lệnh quản trị viên bắt buộc phải thành thạo

Trước và sau khi thay đổi cấu trúc bảng, luôn kiểm tra lại từ điển dữ liệu để xác nhận.

```
-- 1. Xem câu lệnh DDL chi tiết tạo bảng  
SHOW CREATE TABLE products\G
```

```
-- 2. Xem danh sách các cột và kiểu dữ liệu  
DESCRIBE products;  
-- Hoac tương đương:  
SHOW COLUMNS FROM products;
```

```
-- 3. Xem chi tiết các chỉ mục (Index)  
SHOW INDEX FROM products;
```

```
-- 4. Kiểm tra trạng thái bảng trên đĩa  
SHOW TABLE STATUS LIKE 'products'\G
```

Thông tin nhận được từ SHOW CREATE TABLE

Cung cấp chính xác định nghĩa collation, charset, engine, tên các constraint ngoại, check rules và giá trị khởi đầu AUTO_INCREMENT tiếp theo.

6. ALTER TABLE: Thêm, Xóa cột (ADD & DROP)

Thêm cột mới (ADD COLUMN)

- Mặc định cột mới nằm ở cuối bảng.
- Dùng FIRST để đưa lên đầu.
- Dùng AFTER col_name để đặt sau một cột chỉ định.

```
-- Thêm vào cuối:
ALTER TABLE products
ADD COLUMN stock_qty INT NOT NULL DEFAULT
0;
```

```
-- Thêm vào vị trí chỉ định:
ALTER TABLE products
ADD COLUMN barcode VARCHAR(30) AFTER
product_name;
```

Xóa cột (DROP COLUMN)

- Xóa hoàn toàn cột và dữ liệu trong cột đó.
- Chú ý kiểm tra xem cột có đang tham gia vào index hoặc foreign key nào không trước khi xóa.

```
-- Xóa cột khỏi bảng:
ALTER TABLE products
DROP COLUMN barcode;
```

7. Sửa đổi cột: MODIFY vs CHANGE vs RENAME

Mệnh đề	Công dụng chính	Có đổi tên cột không?
MODIFY COLUMN	Sửa kiểu dữ liệu, độ dài, thuộc tính NULL/DEFAULT của cột hiện tại	Không
CHANGE COLUMN	Sửa kiểu dữ liệu đồng thời đổi tên cột	Có
RENAME COLUMN	Chỉ đổi tên cột, giữ nguyên kiểu dữ liệu (từ MySQL 8.0+)	Có

```
-- 1. MODIFY: Mở rộng độ dài từ 100 lên 200
ALTER TABLE products
MODIFY COLUMN product_name VARCHAR(200) NOT
NULL;
```

```
-- 2. RENAME COLUMN: Chỉ đổi tên cột (MySQL
8.0+)
ALTER TABLE products
RENAME COLUMN unit_price TO price;
```

```
-- 3. CHANGE: Vừa đổi tên vừa sửa kiểu
-- Cu pháp: CHANGE old_name new_name
data_type ...
ALTER TABLE products
CHANGE COLUMN price selling_price DECIMAL
(12,2) NOT NULL;
```

8. Quản lý Chỉ mục & Ràng buộc trong ALTER TABLE

Thêm và gỡ bỏ ràng buộc toàn vẹn

Tối ưu hóa bảng trong quá trình vận hành mà không cần phải xóa bảng tạo lại.

```
-- 1. Thêm ràng buộc UNIQUE
ALTER TABLE products
ADD CONSTRAINT uq_products_sku UNIQUE (sku);

-- 2. Thêm ràng buộc CHECK (MySQL 8.0.16+)
ALTER TABLE products
ADD CONSTRAINT chk_products_selling_price CHECK (selling_price >= 0);

-- 3. Thêm chỉ mục phụ (Secondary Index) giúp tăng tốc tìm kiếm
ALTER TABLE products
ADD INDEX idx_products_category (category_id);

-- 4. Xóa ràng buộc Check hoặc Index
ALTER TABLE products DROP CHECK chk_products_selling_price;
ALTER TABLE products DROP INDEX idx_products_category;
```

9. Đổi tên bảng an toàn với RENAME TABLE

Đổi tên đơn lẻ & Chuyển Database

- Đổi tên nhanh chóng ở mức metadata.
- Có thể di chuyển bảng sang database khác trong cùng MySQL instance.

```
-- Đổi tên trong cùng schema:
RENAME TABLE old_products TO products;

-- Chuyển bảng sang schema khác:
RENAME TABLE current_db.products
TO archive_db.products_2026;
```

Atomic Swap: Tráo đổi 2 bảng tức thời

- Hoán đổi bảng cũ và bảng mới trong **một thao tác nguyên tử (atomic)**.
- Ứng dụng triển khai Zero-Downtime Migration!

```
-- Hoán đổi tức thời không làm gián đoạn hệ thống:
RENAME TABLE
products TO products_old,
products_new TO products;
```

10. Xóa bảng: DROP TABLE an toàn

Nguyên lý xóa bảng vật lý

Lệnh DROP TABLE xóa vĩnh viễn định nghĩa cấu trúc bảng, toàn bộ dữ liệu, chỉ mục, trigger và phân quyền liên quan. Dung lượng không gian đĩa được trả về cho hệ điều hành.

```
-- Xoa an toan tranh loi khi bang khong ton tai:  
DROP TABLE IF EXISTS order_lines, products;
```

Quy tắc thứ tự Dependency quan trọng

Khi hai bảng có quan hệ Khóa ngoại (FOREIGN KEY):

- **Thứ tự DROP:** Bắt buộc phải xóa **Bảng con** (order_lines) trước **Bảng cha** (products).
- Nếu cố tình xóa bảng cha trước, MySQL sẽ từ chối và báo lỗi vi phạm ràng buộc tham chiếu (*Cannot drop table referenced by a foreign key*)

11. Bảng tạm: CREATE TEMPORARY TABLE

Đặc điểm của Temporary Table

- **Phạm vi Session:** Chỉ hiển thị duy nhất với kết nối (session) tạo ra nó. Các kết nối khác hoàn toàn không thấy.
- **Tự động giải phóng:** Tự động bị xóa khi kết nối đóng hoặc đứt mạng.
- **Cơ chế Shadowing:** Bảng tạm có thể trùng tên với bảng thật. Khi đó, truy vấn sẽ ưu tiên trở vào bảng tạm!

```
-- Tao bang tam luu ket qua tinh toan
CREATE TEMPORARY TABLE temp_top_customers (
    customer_id INT PRIMARY KEY,
    total_spent DECIMAL(12,2)
);

-- Do du lieu trung gian
INSERT INTO temp_top_customers
SELECT customer_id, SUM(unit_price * quantity
)
FROM order_lines
GROUP BY customer_id;

-- Xoa chu dong khi dung xong
DROP TEMPORARY TABLE temp_top_customers;
```

12. So sánh toàn diện: TRUNCATE TABLE vs DELETE FROM

Tiêu chí	DELETE FROM table_name	TRUNCATE TABLE table_name
Bản chất Cơ chế	Là lệnh DML (Thao tác dữ liệu) Quét và xóa từng dòng một (<i>row-by-row</i>), ghi log chi tiết cho từng dòng	Là lệnh DDL (Định nghĩa dữ liệu) Hủy trang dữ liệu và tạo lại file bảng rỗng (<i>drop and recreate</i>)
Tốc độ	Chậm đối với bảng lớn hàng triệu dòng	Cực nhanh , tốc độ không đổi bất kể kích thước bảng
Transaction	Có thể ROLLBACK an toàn trong transaction	Không thể ROLLBACK (gây Implicit Commit)
AUTO_INCREMENT	Giữ nguyên giá trị sequence tiếp theo	Reset giá trị tự tăng về lại giá trị ban đầu (thường là 1)
Triggers	Kích hoạt trigger BEFORE/AFTER DELETE	Không kích hoạt bất kỳ trigger nào
Khóa ngoại	Xóa được nếu thỏa mãn quy tắc FK	Bị chặn nếu bảng bị tham chiếu bởi FK còn hiệu lực

13. Cột tự sinh: Generated Columns (VIRTUAL vs STORED)

VIRTUAL (Mặc định)

- Không lưu giá trị trên đĩa; giá trị được tính toán **khi đọc**.
- Không tốn dung lượng lưu trữ.
- Có thể tạo Secondary Index trên cột Virtual!

STORED

- Giá trị được tính toán và **lưu cố định xuống đĩa** khi ghi dòng.
- Tốn thêm dung lượng lưu trữ nhưng đọc ra cực nhanh.

```
CREATE TABLE sales_items (  
  item_id      INT PRIMARY KEY AUTO_INCREMENT,  
  quantity     INT NOT NULL,  
  unit_price   DECIMAL(10,2) NOT NULL,  
  
  -- Cột sinh tự động: Total = quantity * unit_price  
  total_amount DECIMAL(12,2) GENERATED ALWAYS AS (quantity * unit_price) VIRTUAL  
) ENGINE = InnoDB;
```

14. Checklist an toàn khi chạy DDL Migration trên Production

Trước khi thực thi DDL

1. **Kiểm tra Database:** Chạy `SELECT DATABASE();` để chắc chắn đúng môi trường.
2. **Backup DDL:** Lưu lại output của lệnh `SHOW CREATE TABLE`.
3. **Đo lường thời gian chạy:** Thử nghiệm trên dữ liệu staging cùng kích thước để đánh giá thời gian lock.
4. **Rà soát phụ thuộc:** Kiểm tra Foreign Key, View, Triggers, Stored Procedure và code ứng

Trong và sau khi thực thi

5. **Chọn thời điểm vắng tải:** Chạy vào khung giờ thấp điểm (off-peak hours) để tránh tắc nghẽn connection.
6. **Online DDL:** Tận dụng cơ chế `ALGORITHM=INPLACE, LOCK=NONE` của InnoDB nếu được hỗ trợ.
7. **Kiểm chứng kết quả:** Dùng `SHOW COLUMNS` và kiểm tra ứng dụng đọc ghi bình thường.
8. **Kế hoạch phục hồi:** Luôn có script rollback chuẩn bị sẵn

15. Câu hỏi ôn tập & Củng cố

Câu 1: Rollback DDL

Lệnh nào sau đây CÓ THỂ được khôi phục bằng lệnh ROLLBACK?

- A. TRUNCATE TABLE
- B. ALTER TABLE
- C. **DELETE FROM** (Chính xác!)
- D. DROP TABLE

Câu 2: Đổi tên cột trong MySQL 8.0

Lệnh nào dùng để chỉ đổi tên cột mà không cần gõ lại kiểu dữ liệu?

- A. MODIFY COLUMN
- B. **RENAME COLUMN** (Chính xác!)
- C. CHANGE COLUMN
- D. UPDATE COLUMN

Câu 3: Reset AUTO_INCREMENT

Hỏi: Khi cần làm rỗng một bảng log kiểm thử và đặt lại mã ID tự tăng về 1, nên dùng lệnh nào?

Tổng kết bài học

Nguyên tắc then chốt

- **Bản chất DDL:** Luôn nhớ DDL gây commit tự động và khóa metadata.
- **Tiến hóa an toàn:** Sử dụng ALTER TABLE cẩn trọng, kiểm tra ràng buộc trước khi sửa đổi.
- **Tối ưu với Generated Columns:** Tránh trùng lặp logic tính toán ở tầng ứng dụng bằng cách chuẩn hóa công thức ngay trong bảng.

Tài liệu học tập tiếp theo

- Chuyên đề tiếp theo: **Truy vấn dữ liệu nâng cao với SELECT** (JOIN, GROUP BY, Subquery, Window Functions).
- Thực hành lab: Hoàn thành toàn bộ kịch bản lifecycle trong schema `table_lifecycle_lab`.